

KI-Netzwerk Schweinfurt

Offizielle **statische Website** des KI-Netzwerks Schweinfurt — Vernetzung von Unternehmen, Bildung, Verwaltung und Community rund um praxisnahe KI in der Region Mainfranken.

Inhalte kommen aus **JSON-Dateien** unter **data/** (Events, Blog, mehrsprachige Seitentexte). Es gibt **kein Build-Tool** und **keine Datenbank**; Pflege erfolgt durch Bearbeiten der Dateien.

Version	0.1.0 (Beta)
Entwicklung	seit April 2026
Entwickler	Oliver Braun — github.com/olli-br

Inhalt

- 1. [Schnellstart](#)
 - 2. [Datenstruktur](#)
 - 3. [Metadaten in Dateien](#)
 - 4. [Neues Event](#)
 - 5. [Neuer Blogbeitrag](#)
 - 6. [Seitentexte & UI-Strings](#)
 - 7. [Schema: Event](#)
 - 8. [Schema: Blog](#)
 - 9. [Konventionen](#)
 - 10. [Projektdateien](#)
 - 11. [Deployment](#)
-

Schnellstart

Zum lokalen Testen im **Projektroot** einen einfachen HTTP-Server starten, damit **fetch** für HTML-Partials und JSON zuverlässig funktioniert:

```
cd /pfad/zu/KI_Netzwerk_Schweinfurt
python3 -m http.server 8000
```

Im Browser: **http://localhost:8000** — Start über **index.html**.

Datenstruktur

Bereich	Pfad	Zweck
---------	------	-------

Bereich	Pfad	Zweck
Events (Inhalte)	<code>data/event/events/*.json</code>	Einzelne Veranstaltungen
Event-Liste	<code>data/event/index.json</code>	Dateinamen der Event-JSONs (<code>files</code>)
Events-UI	<code>data/event/events-page.json</code>	Überschriften, Filter, Event-Detail-Labels (DE/EN)
Blog (Beiträge)	<code>data/blog/posts/*.json</code>	Einzelne Artikel
Blog-Liste	<code>data/blog/index.json</code>	Dateinamen der Post-JSONs (<code>files</code>)
Blog-UI	<code>data/blog/blog-page.json</code>	Überschriften, Liste, Detail (DE/EN)
Startseite	<code>data/start/start-page.json</code>	Hero & Start-Texte (<code>home.*</code> in der App)
Über uns	<code>data/about/about-page.json</code>	About-Texte, Chips, Partner
Vorlagen	<code>data/demo-files/</code>	Leere Schemas zum Kopieren (<code>*-entry.empty.json</code>)

Die App lädt u. a.:

- `./data/event/index.json` → danach je `./data/event/events/<name>`
- `./data/blog/index.json` → danach je `./data/blog/posts/<name>`
- `./data/event/events-page.json`, `./data/blog/blog-page.json`, `./data/start/start-page.json`, `./data/about/about-page.json`

Technik: Vanilla JavaScript (`assets/js/main.js`), Übersetzungs-Fallbacks in `assets/js/i18n.js`, Layout in `assets/css/styles.css`. HTML-Fragmente unter `components/` werden per `fetch` eingebunden und für Karten/Templates genutzt.

Metadaten in Dateien

Zur **Nachverfolgung und Pflege** tragen alle relevanten Quell- und Datendateien einheitliche Metadaten:

Art	Inhalt (oben in der Datei)
HTML (<code>*.html</code> , <code>components/*.html</code>)	HTML-Kommentar: Projekt, Seite , Projekt-Pfad , Beschreibung, Version, Update-Historie, Urheberhinweis
CSS / JS	Blockkommentar: Projekt, Datei , Seite/Rolle , Projekt-Pfad , Version, Historie
JSON unter <code>data/</code>	Root-Objekt <code>_meta</code> : u. a. <code>project</code> , <code>pageName</code> , <code>pathInProject</code> , <code>developedBy</code> , <code>github</code> , <code>developmentStart</code> , <code>currentVersion</code> , <code>updateHistory</code>

Wichtig: `_meta` wird von der **Laufzeit-Logik nicht ausgewertet** — die Seite nutzt weiterhin Felder wie `files`, `title`, `slug`, `id` usw. Du kannst `_meta` bei neuen Einträgen aus einer Vorlage kopieren und **Datum / Beschreibung in `updateHistory`** anpassen, ohne die Funktion der Website zu beeinträchtigen.

Neues Event

- 1. Eine Datei aus `data/event/events/` kopieren (z. B. `event-ai-in-practice.json`).
- 2. Neuen Dateinamen vergeben, z. B. `event-mein-thema.json` (nur Kleinbuchstaben, Zahlen, Bindestrich).
- 3. JSON anpassen: `id`, `date`, `title`, `location`, `geo`, `detail`, ... (siehe [Schema: Event](#)); `_meta` bei Bedarf aktualisieren (`pageName`, `pathInProject`, `updateHistory`).
- 4. Dateinamen in `data/event/index.json` unter `"files"` **ergänzen** (Reihenfolge = Anzeige-Reihenfolge).

Alternativ: `data/demo-files/event-entry.empty.json` als Startpunkt.

Sichtbar: Startseite (Kalender, Karten, Countdown-Leiste), `events.html` (Liste & Detail), DE/EN über Sprachumschaltung.

Neuer Blogbeitrag

- 1. Eine Datei aus `data/blog/posts/` kopieren (z. B. `post-erstes-treffen.json`).
- 2. Neuen Dateinamen vergeben, z. B. `post-mein-artikel.json`.
- 3. JSON anpassen: `slug` muss zur URL passen (`blog.html?post=<slug>`), siehe [Schema: Blog](#); `_meta` bei Bedarf anpassen.
- 4. Dateinamen in `data/blog/index.json` unter `"files"` **ergänzen**.

Alternativ: `data/demo-files/post-entry.empty.json` als Startpunkt.

Sichtbar: Startseite (neueste Beiträge), `blog.html` (Liste & Detail), DE/EN.

Seitentexte & UI-Strings

Datei	Inhalt
<code>data/start/start-page.json</code>	Startseite (dynamische Keys für <code>home.*</code> in der Logik)
<code>data/about/about-page.json</code>	Über uns: Hero, Treffenbeschreibung (<code>meetingBody</code> mit <code>\n\n</code> für Absätze), Teilnehmer- und Partner-Chips als Arrays von <code>{ "de", "en" }</code>
<code>data/event/events-page.json</code>	Events-Seite: Titel, Filter, Badges, Buttons im Detail
<code>data/blog/blog-page.json</code>	Blog: Titel, „Weiterlesen“, Lesezeit-Suffix für Fallback, Zurück-Link

Struktur: pro sichtbarem Text meist Objekte mit `"de"` und `"en"`.

Schema: Event

Feld	Typ	Pflicht	Hinweis
<code>id</code>	Zahl oder String	ja	eindeutig
<code>date</code>	String (ISO)	ja	z. B. <code>2026-06-15T18:00:00</code>
<code>image</code>	String	ja	z. B. <code>./assets/images/event-1.svg</code>
<code>link</code>	String	ja	Anmeldung / externe Seite
<code>title</code>	<code>{ de, en }</code>	ja	
<code>location</code>	Objekt	ja	<code>venue { de, en }, street, postalCode, city</code>
<code>geo</code>	<code>{ lat, lng }</code>	ja	für Karten-Einbettung / Links
<code>detail.description</code>	<code>{ de, en }</code>	ja	Kurztext / Karte
<code>detail.content</code>	<code>{ de, en }</code>	ja	Markdown für die Detailseite
<code>recapLink</code>	String	nein	z. B. für vergangene Events

Schema: Blog

Feld	Typ	Pflicht	Hinweis
<code>id</code>	String	ja	eindeutig
<code>slug</code>	String	ja	URL-Parameter <code>?post=</code>
<code>date</code>	String (ISO)	ja	Sortierung & Anzeige
<code>image</code>	String	ja	
<code>title</code>	<code>{ de, en }</code>	ja	
<code>teaser</code>	<code>{ de, en }</code>	ja	Vorschau in der Karte
<code>detail.content</code>	<code>{ de, en }</code>	ja	Markdown im Artikel
<code>readTime</code>	<code>{ de, en }</code>	empfohlen	z. B. <code>3 Min Lesezeit / 3 min read</code>

Lesezeit: Fehlt `readTime` oder ist er leer, schätzt die Seite die Dauer aus Teaser + Fließtext und nutzt den Text aus `blog-page.json` unter `list.readTime` (Einheit DE/EN).

Konventionen

- **Keine doppelten `id`** (Events) bzw. `id` / `slug` (Blog).
- **Dateinamen:** nur `a-z`, `0-9`, Bindestrich; Endung `.json`.
- **Sprache:** überall dort, wo es vorkommt, `de` und `en` parallel pflegen.
- **`_meta` / Dateiköpfe:** bei inhaltlich größeren Änderungen `updateHistory` (JSON) bzw. die Update-Zeile in den Kommentaren (HTML/CSS/JS) ergänzen — einheitlich mit Datum und Kurzbeschreibung.

Projektdateien

```
KI_Netzwerk_Schweinfurt/
├── index.html           # Startseite
├── events.html          # Events (Liste & Detail)
├── blog.html            # Blog (Liste & Detail)
├── about.html           # Über uns
├── README.md
├── assets/
│   ├── css/styles.css  # Designsystem & Seitenlayout
│   ├── js/main.js      # Daten, Kalender, Blog, Events, Sprache, Theme
│   ├── js/i18n.js      # Statische UI-Strings DE/EN
│   └── images/          # SVGs, Fotos für Events/Blog
├── components/
│   ├── header.html
│   ├── footer.html
│   ├── event-card.html # Template für Event-Karten
│   └── blog-card.html  # Template für Blog-Karten
└── data/
    ├── start/start-page.json
    ├── about/about-page.json
    ├── event/index.json
    ├── event/events-page.json
    ├── event/events/*.json
    ├── blog/index.json
    ├── blog/blog-page.json
    ├── blog/posts/*.json
    └── demo-files/      # Leere Vorlagen
```

Deployment

Dieselbe **reine Static-Site**-Logik eignet sich für **GitHub Pages** oder jeden statischen Host: Root = Webroot, keine Server-API nötig. Wichtig ist, dass Pfade zu JSON und **components/** weiterhin per **fetch** erreichbar sind (gleiche relative Struktur wie lokal).

*Projekt eigenständig entwickelt. Code gerne zum Lernen nutzen — bitte die Attribution in den Dateiköpfen und in **_meta** respektieren.*